WEB DEVELOPMENT INTERNSHIP PROGRAM

FIVE FRONT-END WEB DEVELOPMENT PROJECTS

AN INTERNSHIP PROJECT REPORT

Submitted by

RAGUL A

(Reg No:110725102199)

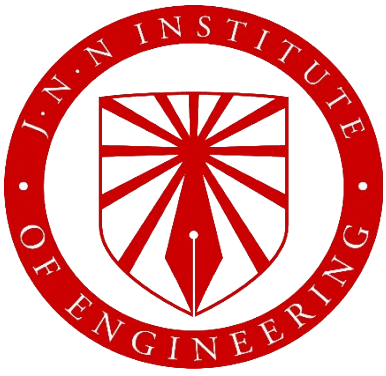
in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

IN

Artificial Intelligence & Data Science



J.N.N Institute of Engineering

(An Autonomous Institute , Affiliated to Anna University, Chennai)

SEPTEMBER – 2026

VISION OF THE INSTITUTE:

Lead the transformation of engineering and management learning experience to educate the next generation of innovators and entrepreneurs who want to make the world a better place.

MISSION OF THE INSTITUTE:

IM1: To develop the required resources and infrastructure and to establish a conducive ambience for the teaching-learning process.

IM2: To nurture professional and ethical values in the students and to instils in them a spirit of innovation and entrepreneurship.

IM3: To encourage a desire for higher learning and research in the students and to equip them to face global challenges.

IM4: To provide opportunities for students to learn job-relevant skills to make them industry ready.

IM5: To interact with industries and other organization to facilitate transfer of knowledge and know-how.

VISION OF THE DEPARTMENT:

To impart quality-education, inculcate professionalism and enhance the problem-solving skills of the students in the domain of Artificial Intelligence & Data Science by applying recent technological tools and incorporating collaborative principles with a focus to make them industry ready.

MISSION OF THE DEPARTMENT:

- **DM 1:** To enhance the knowledge of the students with most recent advancements and refresh their insights in the field of Artificial Intelligence and Data Science.
- **DM 2:** To equip the students with strong fundamental concepts, analytical capability, programming and problem-solving skills.
- **DM 3:** To make the students industry ready and to enhance their employability through training, internships and real-time
- **DM 4:** To guide the students to perform research on Artificial Intelligence and Data Science, with the aim to provide solutions to the problems of the industry

PROGRAM OUTCOMES

S.NO	GRADUATE ATTRIBUTE	PROGRAM OUTCOMES
PO1	Engineering knowledge	Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
PO2	Problem analysis	Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO3	Design/Development of solutions	Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
PO4	Conduct investigations of complex problems	Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO5	Modern Tool usage	Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
PO6	The Engineer and society	Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice
PO7	Environment and sustainability	Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8	Ethics	Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
PO9	Individual and team work	Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
PO10	Communication	Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO11	Project management and finance	Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO12	Life-long learning	Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM EDUCATIONAL OBJECTIVES (PEO)

S.No	PROGRAM EDUCATIONAL OBJECTIVES (PEO)
PEO1	To provide graduates with the proficiency to utilize the fundamental knowledge of basic sciences, mathematics, Artificial Intelligence, data science and statistics to build systems that require management and analysis of large volume of data.
PEO2	To enrich graduates with necessary technical skills to pursue pioneering research in the field of AI and Data Science and create disruptive and sustainable solutions for the welfare of ecosystems.
PEO3	To enable graduates to think logically, pursue lifelong learning and collaborate with an ethical attitude in a multidisciplinary team

m

PROGRAM SPECIFIC OBJECTIVES (PSOs)

S.No	PROGRAM SPECIFIC OBJECTIVES (PSOs)
PSO1	Graduates should be able to evolve AI based efficient domain specific processes for effective decision making in several domains such as business and governance domains.
PSO2	Graduates should be able to arrive at actionable Fore sight, Insight , hind sight from data for solving business and engineering problems
PSO3	Graduates should be able to create, select and apply the theoretical knowledge of AI and Data Analytics along with practical industrial tools and techniques to manage and solve wicked societal problems.



J.N.N INSTITUTE OF ENGINEERING
AUTONOMOUS
NAAC 'A' Grade | Approved by AICTE | Affiliated to Anna University

Approved by



Accredited by



BONAFIDE CERTIFICATE

Certified that this internship project report
"FIVE FRONT-END WEB DEVELOPMENT PROJECTS"
(Web Development Internship, Thiranex) is the bonafide work of "RAGUL A"
(Reg No: 110725102199) who carried out the project under my supervision.

Signature of the HOD with date

Dr. P. Nagarajan

Professor and Head,
Department of Artificial Intelligence
& Data Science Engineering,
J.N.N Institute of Engineering, Chennai

Signature of the Supervisor with date

Miss. Brindha . S

Project Supervisor,
Department of Artificial Intelligence
& Data Science Engineering,
J.N.N Institute of Engineering, Chennai

Certified that the above students have attend of viva voce during the exam held on…….

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

First and foremost, we express our sincere gratitude to our Beloved **FOUNDER & CHAIRMAN Shri. S. JAYACHANDRAN B.Sc., B.L.,** and also heartfelt thanks to our **VICE CHAIRMAN Mr. NAVEEN JAYAHANDRAN M.S.,** for providing necessary facilities for the completion of our project.

We thank our **PRINCIPAL Dr. G. MOHANBABU, Ph.D.,** for the encouragement provided with the congenial environment essential for us to complete the project.

We take this opportunity to express our gratitude to our **HEAD OF THE DEPARTMENT Dr. P. NAGARAJAN** and all other faculty members in the Department of ARTIFICIAL INTELLIGENCE AND DATA SCIENCE for giving us continuous encouragement for completing our project.

We extend our sincere thanks to **Thiranex – Skill Development & Future Tech** and its **Founder & CEO, Mr. Hariharan M,** for giving us the opportunity to undergo this Web Development internship, and we sincerely thank our **mentors** and the entire Thiranex team for their guidance, assignments and feedback throughout this online internship.

Finally, we would like to thank our parents and friends for their constant support and encouragement.

ABSTRACT

Every business, college and start-up now needs a website, and almost every data or AI solution finally reaches its user through a web interface. A beginner who wants to become a web developer must therefore learn how HTML, CSS and JavaScript work together to produce pages that are structured, accessible, responsive and interactive. This report presents the work carried out during the Web Development Internship at Thiranex (Skill Development & Future Tech) from 04th August 2026 to 03rd September 2026, which was conducted in the online mode.

The internship was project based and consisted of five assignments, each adding a new layer to the earlier one. The first was an accessible and search-engine-friendly portfolio built with semantic HTML5, ARIA attributes and a skip link. The second was a five-page responsive portfolio using CSS Grid, Flexbox, media queries and a dark-mode toggle. The third was a To-Do List application with create, edit, delete, complete and filter operations and data saved in localStorage. The fourth was a Weather Dashboard that reads live data from a REST API with fetch() and async/await. The fifth was ShopEase, an e-commerce product catalog with search, category filters and a shopping cart.

The work shows that a student who builds complete projects, tests them in the browser and corrects the errors learns the fundamentals of front-end development far better than one who studies each language separately. The five projects together form a portfolio that progresses from a static page to a data-driven web application, and the report ends with the limitations of the work and the back-end, database and deployment skills that should be learned next.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO.
1	**INTRODUCTION**	**1**
	1.1 OVERVIEW	1
	1.2 INTERNSHIP DETAILS	1
	1.3 OBJECTIVES	2
	1.4 OVERVIEW OF THE INTERNSHIP ACTIVITIES	2
	1.5 PROBLEM STATEMENT	2
	1.6 SCOPE OF THE WORK	3
	1.7 ORGANISATION OF THE REPORT	3
2	**LITERATURE REVIEW**	**4**
	2.1 WEB DEVELOPMENT	4
	2.2 SEMANTIC HTML, ACCESSIBILITY AND SEO	4
	2.3 RESPONSIVE DESIGN WITH CSS GRID AND FLEXBOX	4
	2.4 JAVASCRIPT, THE DOM AND BROWSER STORAGE	5
	2.5 REST APIS AND ASYNCHRONOUS JAVASCRIPT	5
	2.6 E-COMMERCE WEB INTERFACES	5
	2.7 TOOLS AND THEIR PURPOSE	5
	2.8 EXISTING APPROACHES AND THEIR LIMITATIONS	6
	2.9 NEED FOR THE APPROACH FOLLOWED	6
3	**METHODOLOGY**	**7**
	3.1 OVERALL APPROACH	7
	3.2 INTERNSHIP SCHEDULE	7
	3.3 WEB DEVELOPMENT WORKFLOW	7
	3.4 PROJECT DEVELOPMENT METHODOLOGY	7
	3.5 METHODOLOGY OF THE WEATHER DASHBOARD	8
	3.6 METHODOLOGY OF STATE MANAGEMENT	8
	3.7 SYSTEM REQUIREMENTS	8
	3.8 FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS	9
	3.9 TESTING AND DEBUGGING METHOD	9
4	**IMPLEMENTATION**	**10**
	4.1 PROJECT 1: ACCESSIBLE AND SEO-FRIENDLY PORTFOLIO WEBSITE	10
	4.2 PROJECT 2: RESPONSIVE PORTFOLIO WEBSITE (CSS GRID, FLEXBOX AND DARK MODE)	12
	4.3 PROJECT 3: TO-DO LIST APPLICATION (JAVASCRIPT LOGIC AND STATE MANAGEMENT)	15
	4.4 PROJECT 4: WEATHER DASHBOARD (ASYNCHRONOUS JAVASCRIPT AND REST API)	18

CHAPTER NO	TITLE	PAGE NO.
	4.5 PROJECT 5: SHOPEASE – E-COMMERCE PRODUCT CATALOG	21
	4.6 TESTING OF THE PROJECTS	25
	4.7 CHALLENGES FACED AND SOLUTIONS	26
5	**RESULTS AND DISCUSSION**	**27**
	5.1 PROJECTS DEVELOPED	27
	5.2 FEATURES IMPLEMENTED	27
	5.3 SKILLS ACQUIRED	27
	5.4 COMPARISON OF THE FIVE PROJECTS	28
	5.5 INTERNSHIP EXPERIENCE	28
	5.6 DISCUSSION	28
6	**CONCLUSION AND FUTURE WORK**	**29**
	6.1 CONCLUSION OF INDIVIDUAL PROJECTS	29
	6.2 OVERALL CONCLUSION	29
	6.3 FUTURE WORK	29
	6.4 SKILLS ACQUIRED	29
	6.5 LIMITATIONS OF THE PRESENT WORK	30
	6.6 BENEFITS OF THE INTERNSHIP	30
	REFERENCES	31
	APPENDIX – INTERNSHIP CERTIFICATE	32

CHAPTER 1
INTRODUCTION

1.1 Overview

Every business, college and start-up today needs a website. A website is the first place where a customer meets a brand, a student finds a course or a recruiter reads a portfolio. Behind every page that opens in a browser there is a developer who structured the content with HTML, styled it with CSS and made it interactive with JavaScript. Web development is therefore one of the most useful practical skills a student of Artificial Intelligence and Data Science can learn, because almost every data or AI solution finally reaches the user through a web interface.

This report presents the work carried out during the Web Development Internship at Thiranex (Skill Development & Future Tech) from 04th August 2026 to 03rd September 2026. The internship was conducted in the online mode. According to the certificate issued by the company, the intern successfully completed an internship in Web Development during this period. The internship was project based: the learning was organised around five practical assignments, each one adding a new concept to what had been learned before.

The five projects completed were: (1) an accessible and search-engine-friendly portfolio website built with semantic HTML; (2) a responsive multi-page portfolio using CSS Grid, Flexbox and a dark-mode toggle; (3) a To-Do List application that demonstrates JavaScript logic, DOM manipulation and state management with localStorage; (4) a Weather Dashboard that fetches live data from a REST API using asynchronous JavaScript; and (5) ShopEase, an e-commerce product catalog with a product grid, search, category filters and a shopping cart.

1.2 Internship Details

Table 1.1 – Details of the internship

Particulars	Details
Name of the Intern	RAGUL A
Register Number	110725102199
Department	Artificial Intelligence & Data Science
Year / Semester	II Year / III Semester (Batch 2025 – 2029)
Institution	J.N.N Institute of Engineering, Chennai
Organization	Thiranex – Skill Development & Future Tech
Role	Web Development Intern
Mode of Internship	Online
Duration	31 days (04.08.2026 to 03.09.2026)
Projects Completed	Five web development projects (Assignments 1, 4, 5 and Tasks 2, 3)
Certificate issued by	Mr. Hariharan M, Founder & CEO, Thiranex
Certificate ID	THX-AUG0426-138

The internship followed a guided, assignment-based programme. The completion certificate issued by Thiranex, which confirms the domain and the period of the internship, is attached in the Appendix.

1.3 Objectives

- To understand how a website is structured and to write clean, semantic HTML5 that is accessible and friendly to search engines.
- To design responsive layouts using CSS3, Flexbox and CSS Grid that work on mobile, tablet and desktop screens.
- To add behaviour to web pages with JavaScript through DOM manipulation, event handling and browser storage.
- To consume a public REST API using fetch(), async/await and error handling, and to display the result dynamically.
- To build a complete multi-page e-commerce style catalog with search, filtering and a shopping cart.
- To develop the habits of a professional developer: organised folders, readable code, testing in the browser and debugging.

1.4 Overview of the Internship Activities

The internship consisted of five connected projects. Each project introduced a new layer of web development on top of the earlier ones:

- Project 1 – Accessible Portfolio (Assignment 1): semantic HTML5, SEO meta tags, ARIA attributes, a skip-navigation link, focus styles and an accessible contact form.
- Project 2 – Responsive Portfolio (Task 2): a five-page portfolio with CSS variables, Flexbox navigation, CSS Grid card layouts, media queries and a dark-mode toggle.
- Project 3 – To-Do List (Task 3): JavaScript logic with create, read, update and delete operations, task filtering, event delegation and localStorage.
- Project 4 – Weather Dashboard (Assignment 4): asynchronous JavaScript with fetch(), JSON parsing, error handling and dynamic rendering of the OpenWeatherMap response.
- Project 5 – ShopEase Product Catalog (Assignment 5): a four-page online shop with a product grid, search box, category buttons, cart sidebar and checkout message.

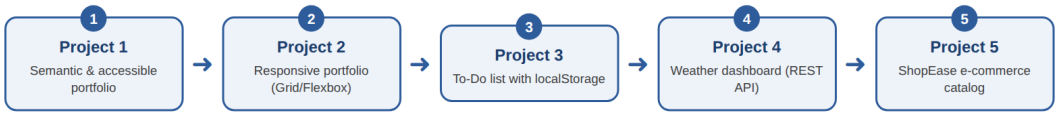


Fig. 1.1 – Phases of the internship work flow

1.5 Problem Statement

A beginner who wants to become a web developer often learns HTML, CSS and JavaScript as separate topics and never joins them into a working product. Pages built this way may look correct on one laptop but fail on a phone, may be unusable with a keyboard or a screen reader, and may

stop working when real data from the internet is involved. Employers, however, expect fresh graduates to deliver complete, tested and user-friendly web applications.

The internship therefore addressed a practical question: how can a student move step by step from a simple static page to an accessible, responsive and data-driven web application, while following good development practice at each stage?

1.6 Scope of the Work

The work was carried out as a learning-oriented internship and is limited to the following scope.

- Included: front-end development of five web projects using HTML5, CSS3 and JavaScript (ES6).
- Included: accessibility, SEO, responsive design, DOM manipulation, browser storage, REST API consumption and cart logic.
- Included: manual testing of every project in the web browser.
- Not included: a server-side back-end, a database, user authentication, a real payment gateway and deployment to a public domain, which are suggested as future work in Chapter 6.

1.7 Organisation of the Report

The report is organised into six chapters. Each chapter builds on the one before it, so that the reader can move from the purpose of the internship to its methods, its execution and finally its outcomes.

- Chapter 1 – Introduction: states the purpose of the internship, the problem, the objectives, the scope and the activities.
- Chapter 2 – Literature Review: explains web development, accessibility, responsive design, JavaScript, REST APIs and e-commerce concepts.
- Chapter 3 – Methodology: describes the approach, the schedule, the development workflow, the requirements and the testing method.
- Chapter 4 – Implementation: presents each project with code, screenshots, tests and the difficulties faced.
- Chapter 5 – Results and Discussion: discusses the outputs, the skills gained and the observations.
- Chapter 6 – Conclusion and Future Work: summarises the work, the skills, the limitations and the future scope.

CHAPTER 2
LITERATURE REVIEW

This chapter gives the background of the concepts, techniques and tools used during the internship.

2.1 Web Development

A website is a collection of web pages that a browser downloads from a server and displays. Every page is built from three core technologies. HTML (HyperText Markup Language) describes the structure and meaning of the content – headings, paragraphs, lists, images, links and forms. CSS (Cascading Style Sheets) controls the appearance – colours, fonts, spacing and layout. JavaScript adds behaviour – menus that open, forms that are checked and content that changes without reloading the page [1], [2], [3].

Front-end development concentrates on what the user sees and touches in the browser. A front-end developer is expected to write code that is readable, that follows web standards and that works across browsers and devices. The textbook approach of keeping structure, presentation and behaviour in separate files – .html, .css and .js – was followed in all five projects of this internship [10].

2.2 Semantic HTML, Accessibility and SEO

Semantic HTML means choosing elements for their meaning rather than their looks. Elements such as header, nav, main, section, article and footer tell the browser, search engines and assistive technologies what each part of the page is. A screen-reader user can jump directly to the main content or the navigation because these landmarks exist.

Accessibility is the practice of making websites usable by everyone, including people who use a keyboard instead of a mouse or a screen reader instead of a monitor. The Web Content Accessibility Guidelines (WCAG) describe the principles of perceivable, operable, understandable and robust content [4]. WAI-ARIA adds attributes such as aria-label, aria-labelledby and aria-current that give extra meaning where plain HTML is not enough [5]. Search Engine Optimisation (SEO) is closely related: descriptive titles, meta descriptions, proper heading order and clean markup help search engines understand and rank a page [12].

2.3 Responsive Design with CSS Grid and Flexbox

Visitors open websites on phones, tablets, laptops and large monitors. Responsive web design makes a single page adapt to all of them by using flexible layouts, relative units and media queries [13]. Flexbox is a one-dimensional layout model that is ideal for navigation bars and rows of buttons because it distributes space and aligns items along one axis. CSS Grid is a two-dimensional model that places items in rows and columns at the same time, which makes it the natural choice for card layouts and page sections [6].

CSS custom properties (variables) allow the colours of a whole site to be defined once and changed in one place. This feature is the basis of the dark-mode switch built in Project 2.

2.4 JavaScript, the DOM and Browser Storage

JavaScript is the programming language of the web. Through the Document Object Model (DOM) it can read, create, change and remove any element of the page. User actions such as clicks and key presses are handled with event listeners. A useful pattern is event delegation, in which one listener on a parent element handles the events of all its children; this keeps the code short and works even for elements created later [3], [11].

Web applications must remember information between visits. The Web Storage API, and in particular localStorage, saves key–value pairs in the browser so that data such as tasks or a theme preference survives a page refresh [7]. Keeping the data in a variable (the application state) and re-drawing the screen from that state after every change is known as state management.

2.5 REST APIs and Asynchronous JavaScript

Many websites show data that does not live inside the page, such as weather, news or prices. This data is obtained from a REST API, which answers an HTTP request with data in JSON format. Because a network request takes time, JavaScript performs it asynchronously: the fetch() function returns a Promise, and the async/await syntax lets the programmer write the steps in a natural order while the page stays responsive [8]. A robust application must also handle failures, for example an unknown city name or a lost connection. In Project 4 the OpenWeatherMap current-weather API was used [9].

2.6 E-Commerce Web Interfaces

An online shop interface has a few standard parts: a navigation bar, a hero banner, a searchable and filterable product grid, a product detail view and a shopping cart that shows the items, the quantity and the total price. Even without a back-end, these parts can be built in the browser with JavaScript arrays and objects, which is a good way to understand how real shopping sites work. Project 5 follows this structure.

2.7 Tools and Their Purpose

Table 2.1 – Tools and technologies used

Tool / Technology	Purpose in the internship
HTML5	Semantic structure of every page, forms and landmarks
CSS3	Layout, colours, typography, Flexbox, Grid, animations and media queries
JavaScript (ES6)	DOM manipulation, events, localStorage, fetch() and cart logic
Visual Studio Code	Code editor used to write and organise the projects
Google Chrome and DevTools	Running pages, inspecting elements, device mode, console and network checks
OpenWeatherMap API	Source of live weather data for the Weather Dashboard
Google Fonts (Poppins)	Web font used in the responsive portfolio
Google Drive	Submission of the completed assignments

2.8 Existing Approaches and Their Limitations

Beginners usually try to create websites in one of the following ways, each with a limitation that the internship approach tried to overcome.

- Website builders and ready-made templates: quick to use, but the learner does not understand the code and cannot customise it deeply.
- Copying finished pages from the internet: produces a result quickly, but the code is often inaccessible, untested and not understood.
- Learning each language in isolation: gives knowledge of syntax but not the ability to combine HTML, CSS and JavaScript into a working application.
- Using large frameworks too early: hides the fundamentals and makes debugging difficult for a fresher.

2.9 Need for the Approach Followed

The internship used a project-based approach in which every assignment was a small, complete product. Starting with plain HTML and CSS and adding JavaScript, an API and a cart step by step made sure that the fundamentals were understood before any tool or framework was introduced. Because every project was tested in the browser and corrected, the intern also learned debugging and the discipline of delivering working code.

CHAPTER 3
METHODOLOGY

3.1 Overall Approach

The internship followed a step-by-step approach in which each project was planned, coded, tested and submitted before the next one began. The first project fixed the basics of semantic markup. The second added professional styling and responsive layout. The third introduced programming logic. The fourth connected the page to the internet through an API, and the fifth combined all earlier skills in a larger product. This order meant that every new project could reuse the knowledge of the previous ones.

3.2 Internship Schedule

The internship ran for 31 days in the online mode. Table 3.1 shows how the work was distributed over the period.

Table 3.1 – Week-wise activities during the internship

Period	Activities
Week 1 04 – 10 Aug	Introduction to the internship; web fundamentals; Project 1 (accessible portfolio) and Project 2 (responsive portfolio)
Week 2 11 – 17 Aug	Project 3: To-Do List application with DOM manipulation, filtering and localStorage
Week 3 18 – 24 Aug	Project 4: Weather Dashboard with fetch(), async/await and error handling
Week 4 25 – 31 Aug	Project 5: ShopEase product catalog with search, categories and cart
Final days 01 – 03 Sep	Testing, correction of errors, review of all projects, preparation of the report and receipt of the certificate

3.3 Web Development Workflow

For every project the same repeatable workflow was followed, as shown in Fig. 3.1. The requirement is understood first, then the design is planned, then the code is written, tested and debugged, and finally the project folder is submitted.

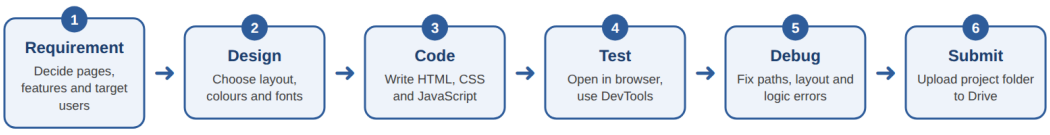


Fig. 3.1 – Web development workflow followed for each project

3.4 Project Development Methodology

- Requirement: decide the purpose of the page, the sections needed and the features expected.
- Design: choose a colour theme, fonts and the layout of every page before writing the code.
- Structure: write the HTML first so that the content is complete and meaningful without any

styling.

- Style: add CSS in an external file, using variables and a mobile-friendly layout.
- Behaviour: add JavaScript in an external file and connect it to the page elements by id or class.
- Test: open every page, click every link and button, resize the window and check the console for errors.
- Submit: save the project in its own folder with a clear structure and upload it to Google Drive.

3.5 Methodology of the Weather Dashboard

The Weather Dashboard needs data from outside the page, so its method differs slightly from the other projects. The user types a city name, the script checks that the field is not empty, sends a request to the weather API, waits for the reply, converts the reply to a JavaScript object and finally copies the required values into the page. If the request fails, a clear message is shown instead of a blank screen. Fig. 3.2 shows these steps.



Fig. 3.2 – Request and response flow of the Weather Dashboard

3.6 Methodology of State Management

The To-Do List and the shopping cart both depend on data that changes while the user works. In both projects the data is kept in an array, every user action modifies the array first, and the page is then drawn again from the array. For the To-Do List the array is also saved in localStorage so that the tasks are still there after a refresh. Fig. 3.3 shows this cycle.



Fig. 3.3 – State management cycle of the To-Do List application

3.7 System Requirements

The work does not need special equipment. Table 3.2 lists the minimum hardware and software used.

Table 3.2 – Minimum hardware and software requirements

Item	Requirement
Processor	Intel Core i3 or equivalent, or higher
Memory	4 GB RAM or more
Operating system	Windows 10/11, Linux or macOS
Browser	Google Chrome (latest version) with Developer Tools
Editor	Visual Studio Code or any text editor

Item	Requirement
Internet	Required for the online sessions, Google Fonts, image links and the weather API
API key	A free OpenWeatherMap API key for the Weather Dashboard

3.8 Functional and Non-Functional Requirements

Functional requirements

- Every website shall have a navigation bar with working links between its pages.
- The accessible portfolio shall provide a skip link, landmarks and a labelled contact form.
- The To-Do List shall create, edit, delete and complete tasks, filter them and keep them after a refresh.
- The Weather Dashboard shall show temperature, humidity, wind speed and condition for a city and show an error for invalid input.
- The ShopEase catalog shall list products, filter them by category and search, and maintain a cart with a total.

Non-functional requirements

- Responsiveness: the pages should adjust to phone, tablet and desktop screens.
- Usability: text, buttons and forms should be easy to read and operate, also from the keyboard.
- Maintainability: HTML, CSS and JavaScript should be kept in separate, well-named files.
- Reliability: the applications should handle wrong or empty input without breaking.

3.9 Testing and Debugging Method

Testing was done manually in the browser. Each page was opened, every link and button was clicked, and the window was resized to phone, tablet and desktop widths with the device mode of Chrome DevTools. The console was watched for errors, and the Elements and Network panels were used to check styles and requests. The projects with logic (To-Do List, Weather Dashboard and ShopEase cart) were additionally tested with a checklist of user actions and expected results, which is given in Chapter 4.

CHAPTER 4
IMPLEMENTATION

This chapter describes the practical execution of each project. For every project the services used, the explanation, the important code and the output screenshots are given.

4.1 Project 1: Accessible and SEO-Friendly Portfolio Website

Services used: HTML5, CSS3, ARIA attributes, Google Chrome

The first assignment was to build a small portfolio website that is correct from the point of view of structure, accessibility and search engines. The site has four pages – Home, About, Projects and Contact – connected by a navigation bar. Instead of using generic div elements, each page uses the header, nav, main, section, article and footer elements, and every section is connected to its heading with aria-labelledby so that a screen reader announces its purpose.

The head of the home page carries the SEO information: the character set, the viewport setting for mobile devices, a meta description, meta keywords, the author and a meaningful page title. A skip-navigation link placed before the header lets keyboard users jump straight to the main content, and the current page in the navigation bar is marked with aria-current="page". The code is shown in Code 4.1.

```
<head>
    <!-- SEO Meta Tags -->
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="Ragul A Portfolio - Web Developer and Data
Enthusiast">
    <meta name="keywords" content="portfolio, web developer, data analyst, projects">
    <meta name="author" content="Ragul A">
    <title>Home | Portfolio</title>

    <link rel="stylesheet" href="css/styles.css">
  ...
```

Code 4.1 – SEO meta tags in the head of index.html

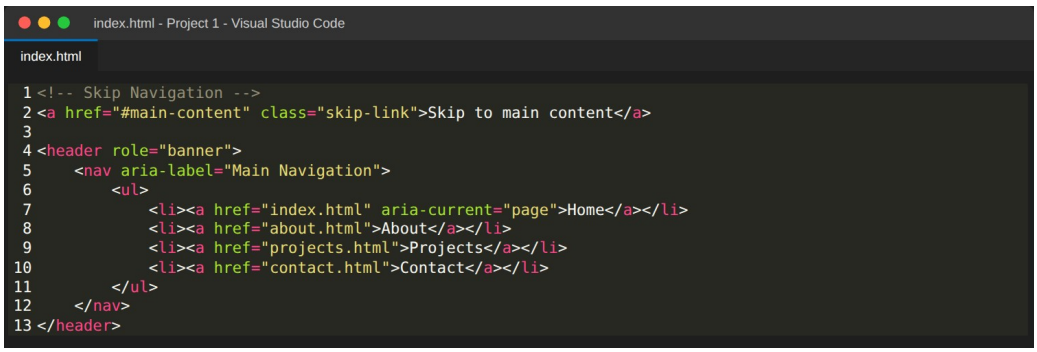


Fig. 4.1 – Skip link and navigation landmarks of the accessible portfolio in the code editor

The style sheet adds the accessibility features that cannot be done with markup alone. The skip link is moved out of sight and appears only when it receives keyboard focus, and a thick blue outline is shown on every link, button and form field that is focused, so that a keyboard user always knows where he or she is. The navigation list is laid out in a row with Flexbox. These rules are shown in Code 4.2.

```
/* Skip link */
.skip-link {
    position: absolute;
    left: -999px;
}
.skip-link:focus {
    left: 10px;
    top: 10px;
    background: black;
    color: white;
    padding: 8px;
}

/* Focus styles */
a:focus, button:focus, input:focus, textarea:focus {
    outline: 3px solid blue;
}
/* Navigation */
nav ul { display: flex; gap: 15px; list-style: none; }
```

Code 4.2 – Skip link and focus styles in styles.css



Fig. 4.2 – Home page of the accessible portfolio website

Fig. 4.3 shows the skip link appearing at the top-left corner when the Tab key is pressed once. This small feature saves keyboard users from moving through the whole navigation on every page.



Fig. 4.3 – Skip link visible when the Tab key is pressed

The contact page contains a form in which every input has a visible label connected by the for and id attributes, the correct input types (text and email) and the required attribute, so that the browser itself validates the fields before submission. The form and its output are shown in Code 4.3 and Fig. 4.4.

```
<form aria-label="Contact Form">

            <label for="name">Name</label>
            <input id="name" name="name" type="text" required>

            <label for="email">Email</label>
            <input id="email" name="email" type="email" required>

            <label for="message">Message</label>
            <textarea id="message" name="message" required></textarea>

            <button type="submit">Send Message</button>


</form>
```

Code 4.3 – Accessible contact form in contact.html



Fig. 4.4 – Contact page with labelled form fields

4.2 Project 2: Responsive Portfolio Website (CSS Grid, Flexbox and Dark Mode)

Services used: HTML5, CSS3 (Grid, Flexbox, variables, media queries), JavaScript, Google Fonts

The second project was a complete five-page portfolio – Home, About, Skills, Projects and Contact – designed to look professional on every screen. All pages share one style sheet and one script. The Poppins font is loaded from Google Fonts, a sticky navigation bar built with Flexbox stays at the top while scrolling, and the hero section is a two-column CSS Grid with the introduction on one side and a circular profile image on the other.

The colours of the whole website are stored as CSS variables on the :root selector. A second set of values is defined for the dark theme under the body.dark selector, so switching the theme means only adding or removing one class. Cards for features, services and projects are placed in grids of three columns, and media queries reduce the columns to two on tablets and one on phones. The important rules are shown in Code 4.4.

```
:root{ --primary:#2563eb; --secondary:#1e293b; --background:#f8fafc;
       --white:#ffffff; --text:#222; }
body.dark{ --primary:#60a5fa; --secondary:#111827; --background:#0f172a;
           --white:#1e293b; --text:#ffffff; }

.hero{ display:grid; grid-template-columns:repeat(2,1fr);
       align-items:center; gap:60px; padding:90px 8%;
       background:linear-gradient(135deg,#2563eb,#7c3aed); }

.feature-container{ display:grid; grid-template-columns:repeat(3,1fr); gap:28px; }

@media(max-width:992px){
  .feature-container{ grid-template-columns:repeat(2,1fr); }
  .hero{ grid-template-columns:1fr; text-align:center; }
}
@media(max-width:600px){
  .feature-container{ grid-template-columns:1fr; }
}
```

Code 4.4 – CSS variables, Grid layout and media queries in style.css

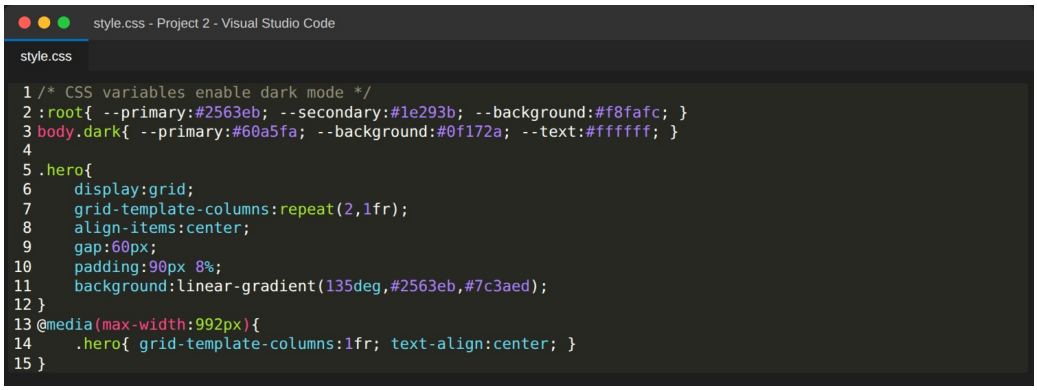


Fig. 4.5 – Responsive CSS of the portfolio in the code editor

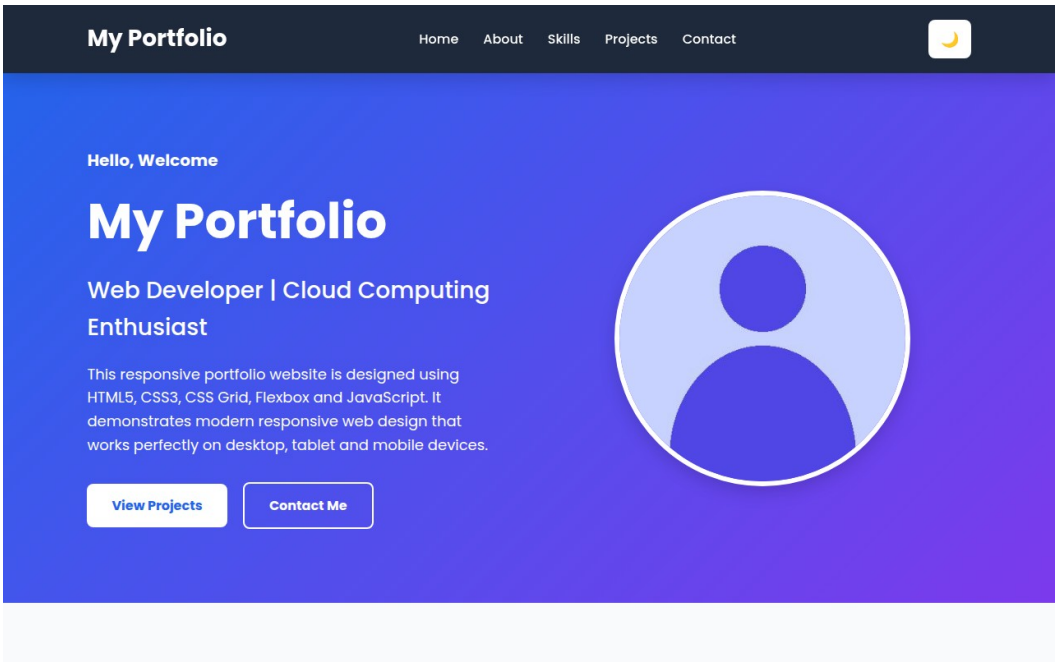


Fig. 4.6 – Home page of the responsive portfolio (desktop view)

The Skills page presents each technology as a card with a progress bar, and the Projects page lists the academic and personal projects as cards with a View Project button. The Skills page is shown in Fig. 4.7.

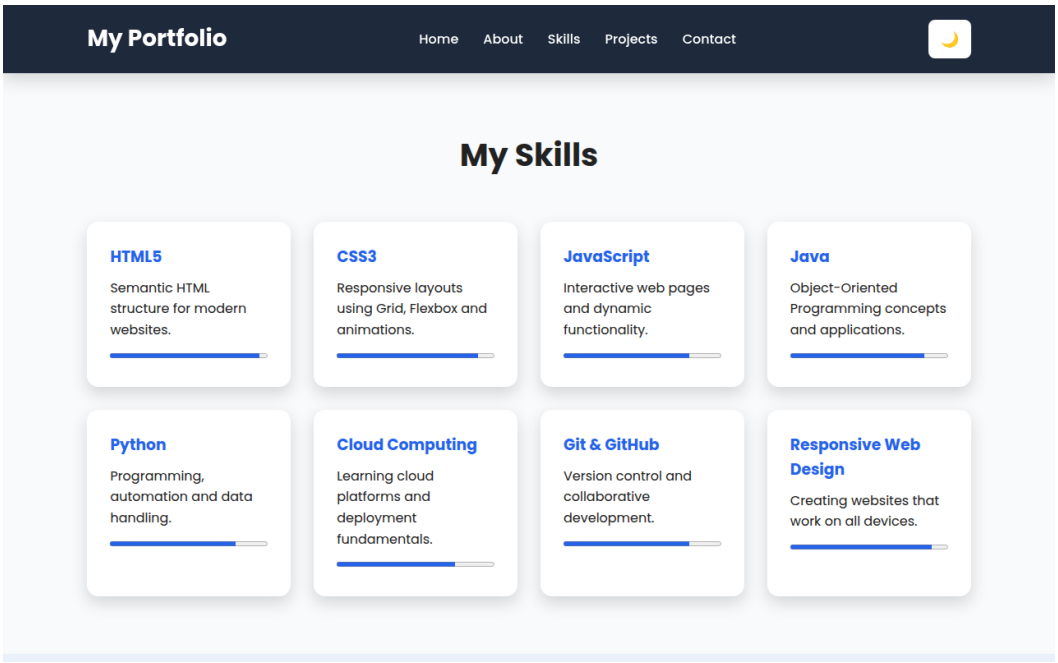


Fig. 4.7 – Skills page with skill cards and progress bars

The theme button in the navigation bar is handled by a short script. When it is clicked, the dark class is toggled on the body, the icon changes between a moon and a sun, and the choice is saved in localStorage so that it is remembered on the next visit (Code 4.5). Fig. 4.8 shows the dark theme, and Fig. 4.9 shows the same page on a phone and on a tablet, where the layout has changed to one and two columns.

```
// Dark / light theme toggle with saved preference
const toggle = document.getElementById("theme-toggle");
if (localStorage.getItem("theme") === "dark") {
    document.body.classList.add("dark");
    toggle.textContent = "☀️";
}
toggle.addEventListener("click", () => {
    document.body.classList.toggle("dark");
    const dark = document.body.classList.contains("dark");
    toggle.textContent = dark ? "☀️" : "🌙";
    localStorage.setItem("theme", dark ? "dark" : "light");
});
```

Code 4.5 – Dark-mode toggle in script.js



Fig. 4.8 – Home page after switching to the dark theme

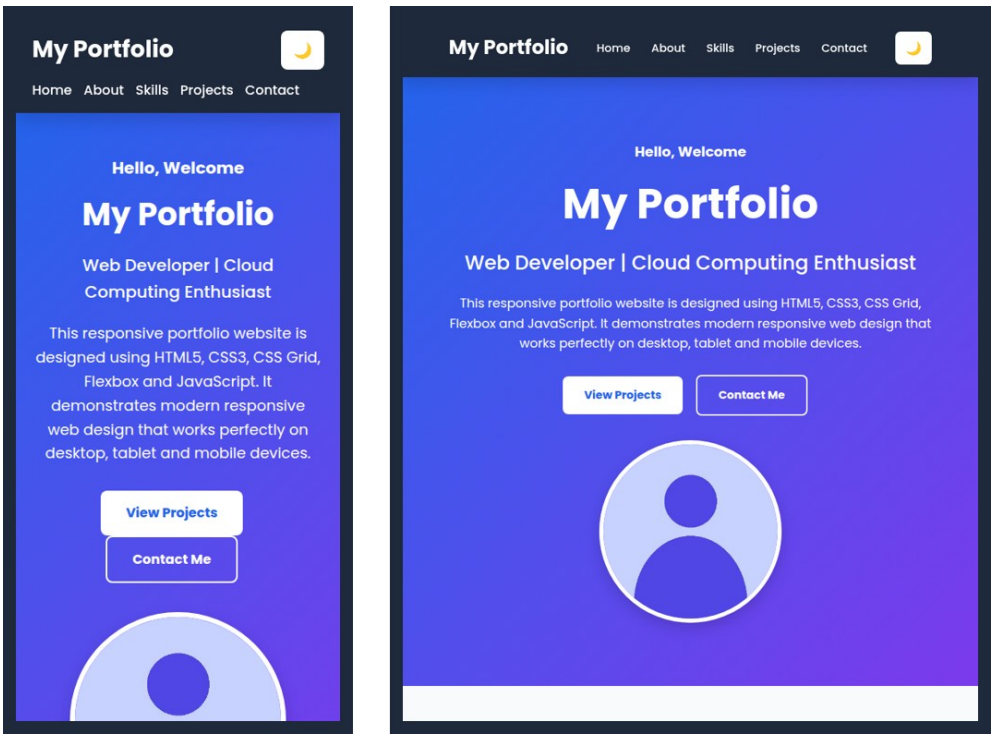


Fig. 4.9 – The portfolio on a phone (left) and on a tablet (right)

4.3 Project 3: To-Do List Application (JavaScript Logic and State Management)

Services used: HTML5, CSS3, JavaScript (ES6), localStorage

The third project is an interactive client-side To-Do List. It moves the intern from designing pages to writing real programme logic. The user can create a task, view all tasks, edit a task, delete a task, mark it as completed and filter the list into All, Active and Completed. All tasks are saved in localStorage, so they are still present after the page is refreshed or the browser is closed.

The application keeps its state in a single array called tasks. Each task is an object with an id, the text and a completed flag. The function renderTasks() clears the list and creates a new list item for every task that matches the current filter, using document.createElement. The add function trims the text, rejects empty input, pushes a new object into the array, saves the array and calls renderTasks(). The core of the script is shown in Code 4.6.

```
let tasks = JSON.parse(localStorage.getItem("tasks")) || [];
let currentFilter = "all";

function saveTasks() {
    localStorage.setItem("tasks", JSON.stringify(tasks));
}

function getVisibleTasks() {
    if (currentFilter === "active") return tasks.filter(t => !t.completed);
    if (currentFilter === "completed") return tasks.filter(t => t.completed);
    return tasks;
}

function renderTasks() {
    taskList.innerHTML = "";
    getVisibleTasks().forEach(task => {
        const li = document.createElement("li");
        li.dataset.id = task.id;
        if (task.completed) li.classList.add("completed");

        const span = document.createElement("span");
        span.textContent = task.text;

        const buttons = document.createElement("div");
        buttons.className = "task-buttons";
        buttons.innerHTML =
            '<button class="edit-btn">Edit</button>' +
            '<button class="delete-btn">Delete</button>';

        li.append(span, buttons);
        taskList.appendChild(li);
    });
}

function addTask() {
    const text = taskInput.value.trim();
    if (text === "") return;
    tasks.push({ id: Date.now(), text: text, completed: false });
    taskInput.value = "";
    saveTasks();
    renderTasks();
}
```

Code 4.6 – State, saving and rendering functions in script.js

Instead of attaching a listener to every Edit and Delete button, a single click listener is attached to the task list. It finds the clicked list item with closest("li"), reads its id from the data-id attribute and decides what to do from the class of the clicked element. This is the event delegation pattern, and it is shown as an editor view in Fig. 4.10.

```
taskList.addEventListener("click", function (e) {
    const li = e.target.closest("li");
    if (!li) return;
    const id = Number(li.dataset.id);

    if (e.target.classList.contains("delete-btn")) {
        tasks = tasks.filter(t => t.id !== id);
    } else if (e.target.classList.contains("edit-btn")) {
        const task = tasks.find(t => t.id === id);
        const updated = prompt("Edit task:", task.text);
        if (updated !== null && updated.trim() !== "") task.text = updated.trim();
    } else if (e.target.tagName === "SPAN") {
        const task = tasks.find(t => t.id === id);
        task.completed = !task.completed;
    }
    saveTasks();
    renderTasks();
});
```

Fig. 4.10 – Event delegation for edit, delete and complete actions

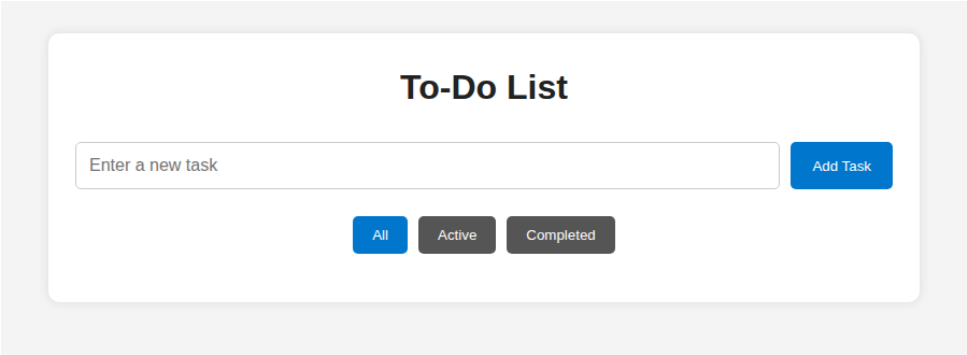


Fig. 4.11 – To-Do List application on first start (no tasks)

After four tasks were added and two of them were clicked to mark them as completed, the list looks as in Fig. 4.12. Completed tasks are shown with a line through the text. Choosing the Completed filter shows only these tasks (Fig. 4.13).

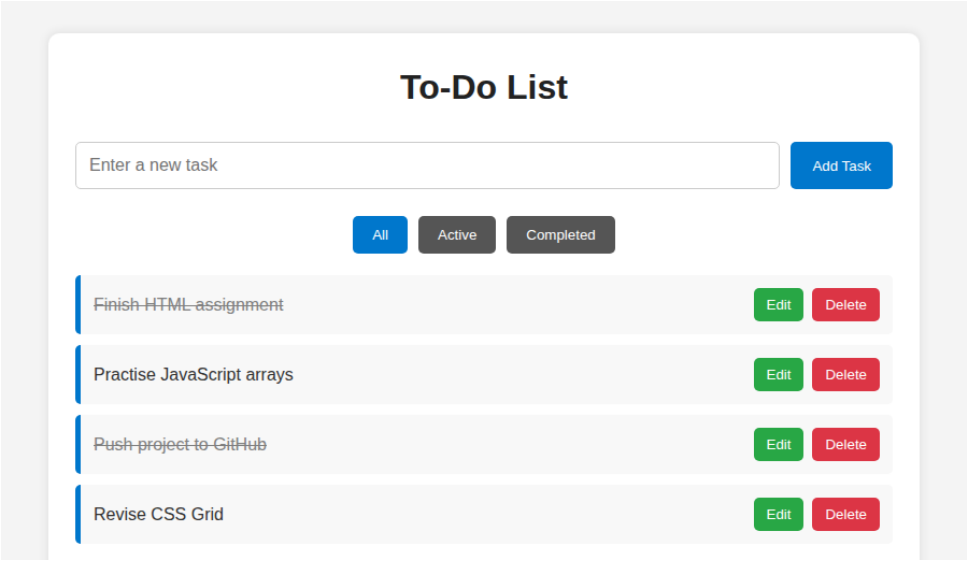


Fig. 4.12 – Task list with active and completed tasks

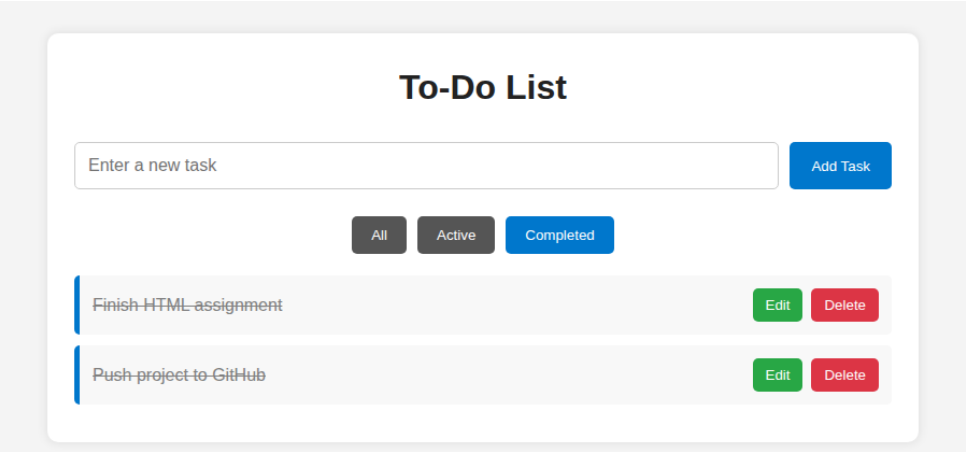


Fig. 4.13 – Completed filter showing only finished tasks

4.4 Project 4: Weather Dashboard (Asynchronous JavaScript and REST API)

Services used: HTML5, CSS3, JavaScript (fetch, async/await), OpenWeatherMap API

The fourth project is a Weather Dashboard. The page has a search box and a button; when a city name is entered the page shows the temperature in degrees Celsius, the humidity, the wind speed and a short description of the weather. The page is designed with a blue gradient background and a translucent card for the result, and a hidden class is used to show the card only after data has been received.

The function getWeather() is declared as async. It first reads the city, clears the old message and hides the old card. If the city is empty, an error message is shown and the function stops. Otherwise it sends a request with fetch() to the current-weather endpoint of OpenWeatherMap with the city, the API key and units=metric. If the server replies with an error status such as 404 for an unknown city, the code throws an Error that is caught and displayed. When the reply is successful, response.json() converts it to an object and the required values are read from the nested structure – data.main.temp, data.main.humidity, data.wind.speed and data.weather[0].description – and written into the page with textContent. The complete function is given in Code 4.7.

```
async function getWeather() {
    const city = document.getElementById("cityInput").value;
    const errorDiv = document.getElementById("error");
    const weatherCard = document.getElementById("weatherResult");

    errorDiv.textContent = "";
    weatherCard.classList.add("hidden");

    if (!city) {
        errorDiv.textContent = "Please enter a city name!";
        return;
    }

    try {
        const response = await fetch(
            `https://api.openweathermap.org/data/2.5/weather?q=${city}&appid=$
{apiKey}&units=metric`
        );

        // Error Handling
        if (!response.ok) {
            throw new Error("City not found or API error!");
        }

        const data = await response.json();

        // Parsing Nested JSON
        const temperature = data.main.temp;
        const humidity = data.main.humidity;
        const windSpeed = data.wind.speed;
        const condition = data.weather[0].description;

        // Dynamic Rendering
        document.getElementById("cityName").textContent = data.name;
        document.getElementById("temp").textContent = temperature;
        document.getElementById("humidity").textContent = humidity;
        document.getElementById("wind").textContent = windSpeed;
        document.getElementById("condition").textContent = condition;

        weatherCard.classList.remove("hidden");

    } catch (error) {
        errorDiv.textContent = error.message;
    }
}
```

Code 4.7 – The getWeather() function of the Weather Dashboard

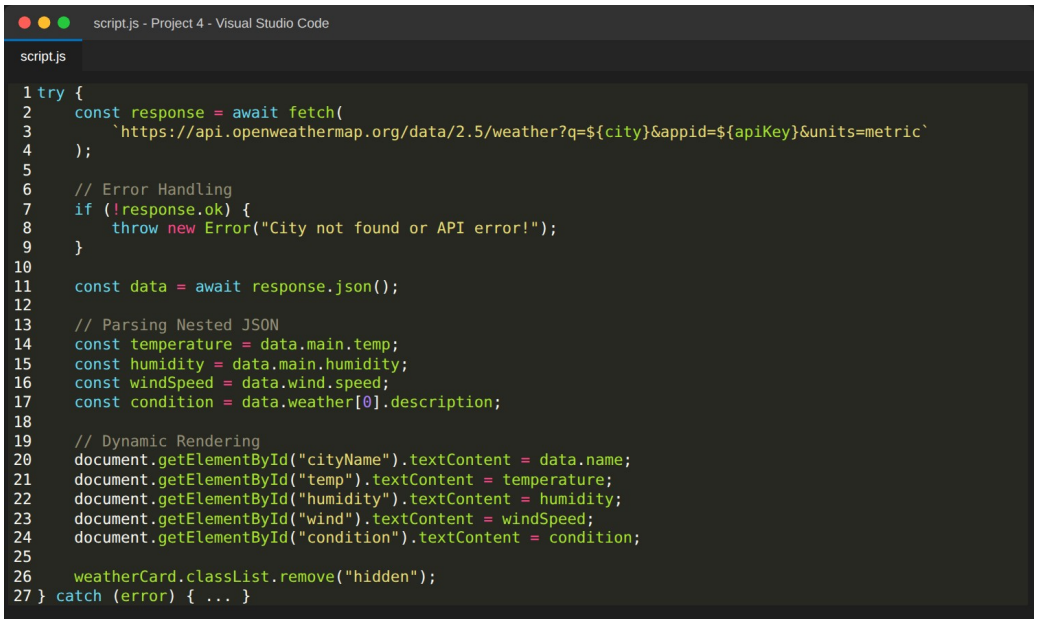


Fig. 4.14 – Fetch request, error handling and JSON parsing in the code editor

Fig. 4.15 shows the dashboard before any search. When the Search button is clicked with an empty box, the message “Please enter a city name!” appears (Fig. 4.16). When a valid city is entered, the weather card appears (Fig. 4.17), and for an unknown city the error message “City not found or API error!” is displayed (Fig. 4.18).

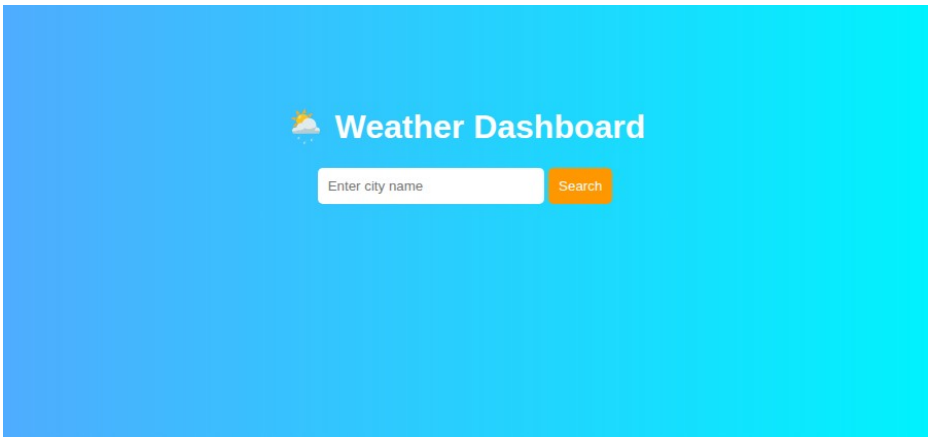


Fig. 4.15 – Weather Dashboard on first load

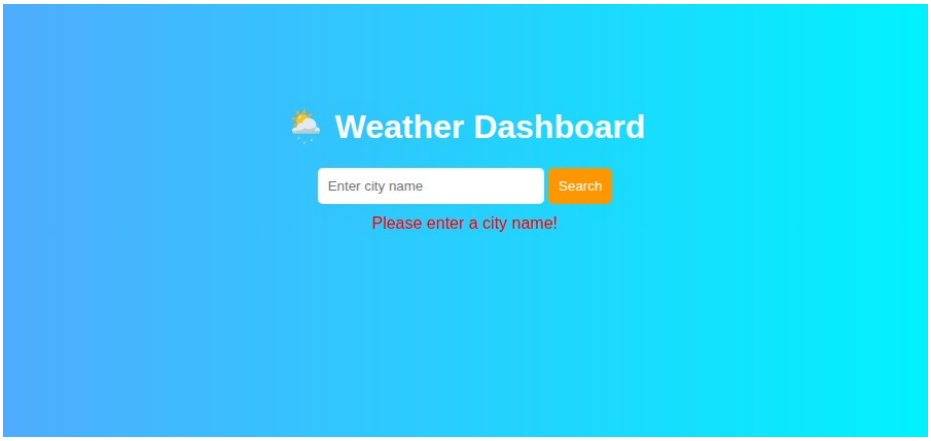


Fig. 4.16 – Validation message for an empty search

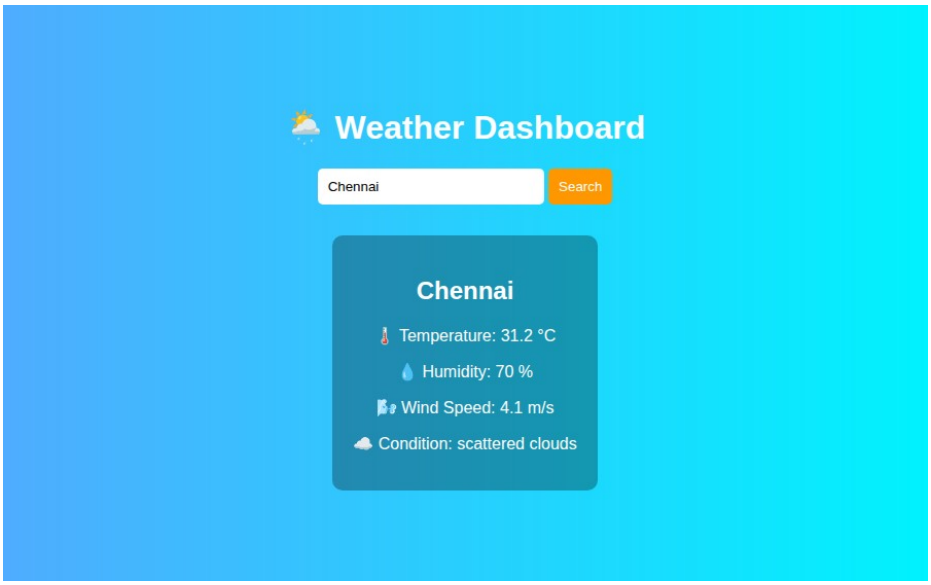


Fig. 4.17 – Weather card displayed for a valid city

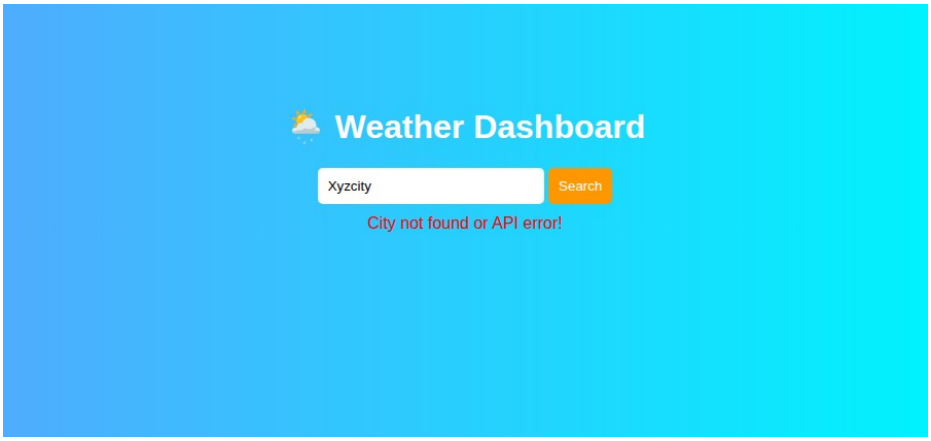


Fig. 4.18 – Error message displayed for an unknown city

4.5 Project 5: ShopEase – E-Commerce Product Catalog

Services used: HTML5, CSS3, JavaScript (ES6), Chrome DevTools

The fifth and largest project is ShopEase, an online shopping website. It is organised as a small multi-page site with the pages Home, Products, About and Contact, a shared style sheet in assets/css and the JavaScript split into modules in assets/js. Splitting the script into products.js (data), cart.js (cart logic), router.js (page switching), ui.js (drawing the screen) and app.js (event wiring) kept every file short and gave each file one responsibility (Fig. 4.19).

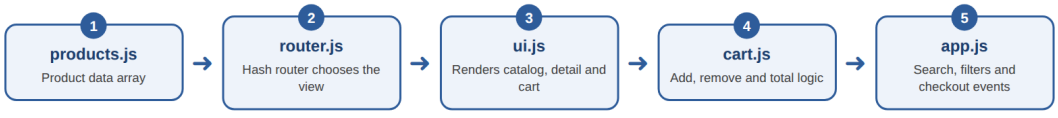


Fig. 4.19 – Modules of the ShopEase JavaScript code

The main page has a sticky blue navigation bar with a cart button and a live item counter, a hero banner with a Shop Now button, a search box and category buttons for All, Electronics, Fashion, Accessories and Shoes. Products are stored in an array of objects with an id, name, price, category, rating and image. When the user types in the search box or clicks a category, the array is filtered

and the grid is drawn again by renderCatalog(), which turns every product into a card with its image, name, price in rupees, star rating and an Add to Cart button (Code 4.8).

```
function renderCatalog(list) {
    if (list.length === 0) {
        app.innerHTML = '<p style="text-align:center;padding:40px">No products
found.</p>';
        return;
    }
    app.innerHTML = '<div class="products-grid">' + list.map(p => `
        <div class="product-card">
            <a href="#/product/${p.id}"><img src="${p.image}" alt="${p.name}"></a>
            <div class="product-info">
                <h3>${p.name}</h3>
                <div class="price">${rupee(p.price)}</div>
                <div class="rating">${"★".repeat(p.rating)}${"☆".repeat(5 -
p.rating)}</div>
                <button class="add-cart" onclick="addToCart(${p.id})">Add to
Cart</button>
            </div>
        </div>`).join("") + "</div>";
}
```

Code 4.8 – Product grid rendering in ui.js



Fig. 4.20 – renderCatalog() function in the code editor

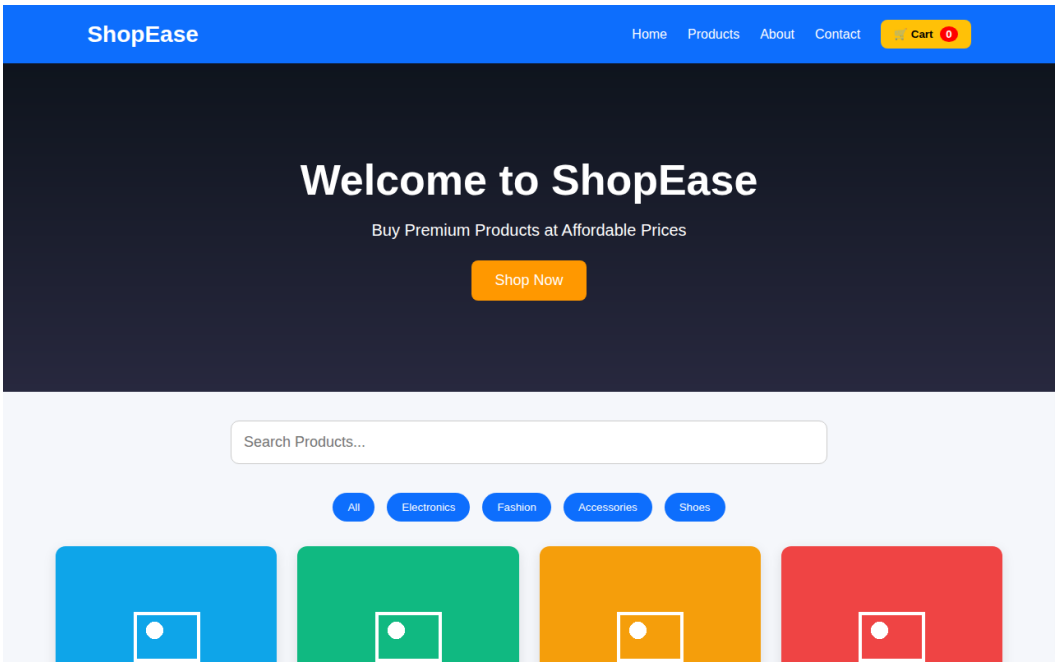


Fig. 4.21 – ShopEase home page with hero banner, search and categories

The shopping cart is a sidebar that slides in from the right. addToCart() increases the quantity if the product is already in the cart or adds a new entry; cartTotal() multiplies each price by its quantity

and adds the results; and renderCart() shows the items, the total and the number on the cart button. Pressing Checkout shows a confirmation message and empties the cart (Code 4.9). Fig. 4.22 shows the cart after two different products were added.

```
// Shopping cart logic
let cart = [];

function addToCart(id) {
    const item = cart.find(c => c.id === id);
    if (item) { item.qty += 1; }
    else { cart.push({ id: id, qty: 1 }); }
    renderCart();
}

function removeFromCart(id) {
    cart = cart.filter(c => c.id !== id);
    renderCart();
}

function cartTotal() {
    return cart.reduce((sum, c) => {
        const p = products.find(x => x.id === c.id);
        return sum + p.price * c.qty;
    }, 0);
}
```

Code 4.9 – Cart logic in cart.js

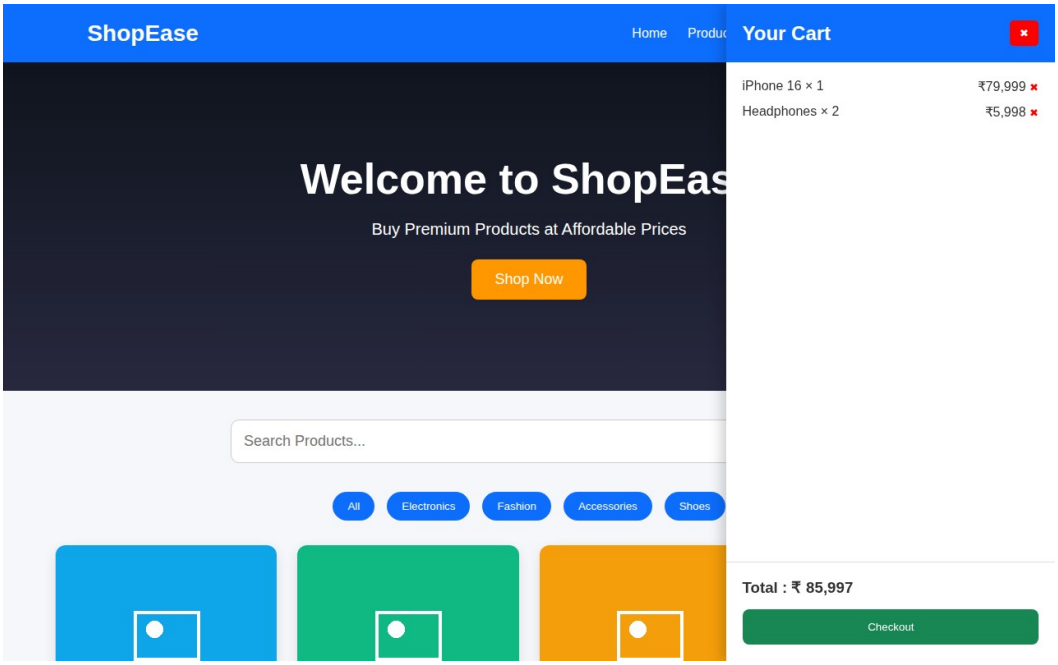


Fig. 4.22 – Cart sidebar with items, quantity and total

The separate pages of the site – Products, About and Contact – use the same navigation bar and style sheet. The Products page lists four items in a card grid, the About page explains the idea and mission of the shop, and the Contact page contains a form with name, e-mail and message fields. Fig. 4.23 shows the Products page.

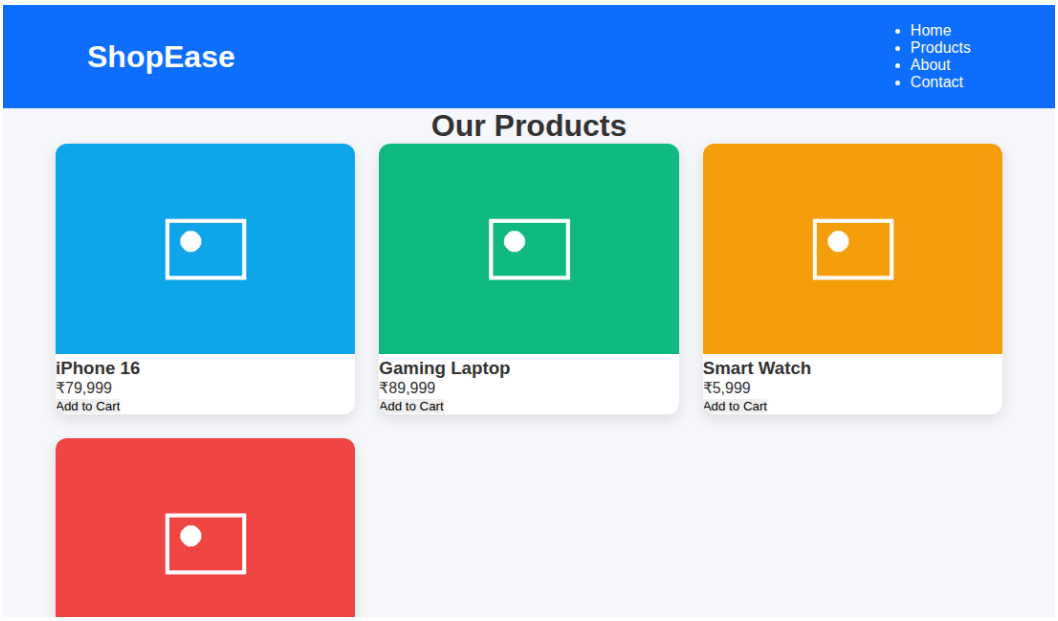


Fig. 4.23 – Products page of ShopEase

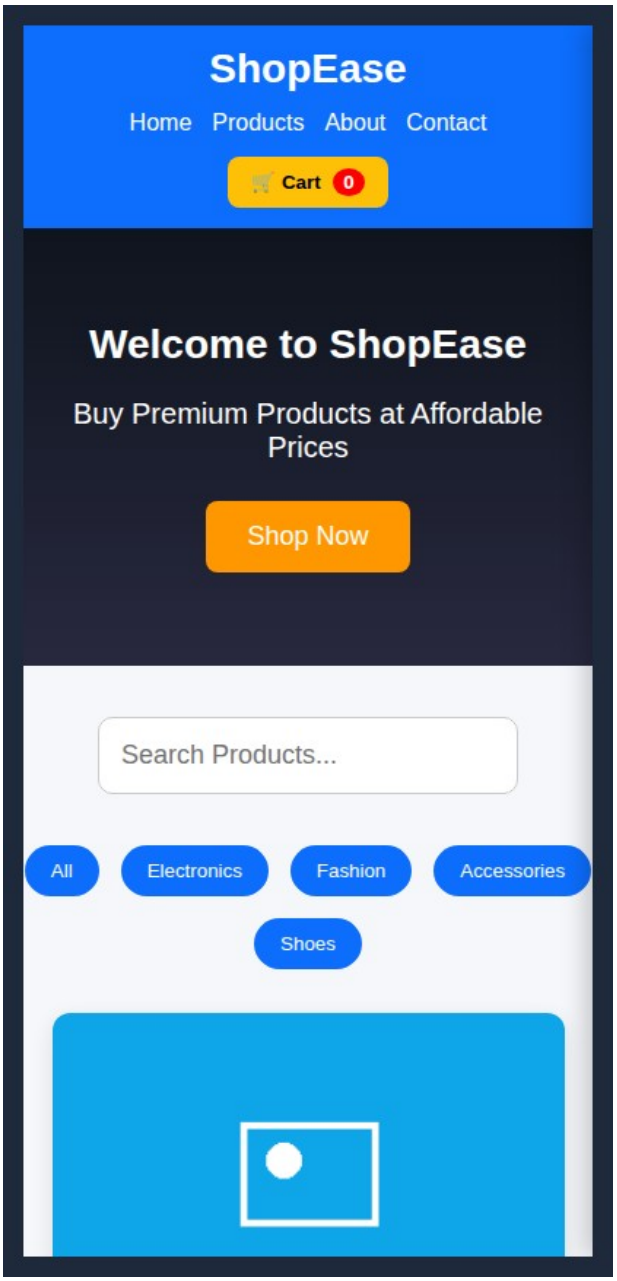


Fig. 4.24 – ShopEase on a phone-sized screen

4.6 Testing of the Projects

Each project was tested in the browser with the checklist in Table 4.1. The To-Do List, the Weather Dashboard and the ShopEase cart were also run through scripted browser tests that repeat the user actions and compare the result with the expected result; the weather requests in that test were answered with sample data so that the test did not depend on an API key.

Table 4.1 – Test checklist

Project	Check performed	Result
Project 1	Tab key shows the skip link; focus outline visible on links and fields	Passed
Project 1	Contact form refuses an empty name or an invalid e-mail address	Passed
Project 2	Navigation links open the correct pages; theme toggle changes and remembers the theme	Passed

Project	Check performed	Result
Project 2	Layout changes to two columns at 992 px and one column at 600 px	Passed
Project 3	Create: four tasks added to the list	Passed
Project 3	Update: task text changed with the Edit button	Passed
Project 3	Complete: clicking a task marks it with a line through the text	Passed
Project 3	Filters: Active and Completed show only the matching tasks	Passed
Project 3	Delete: removed task disappears from the list	Passed
Project 3	Persistence: tasks are still present after the page is refreshed	Passed
Project 3	Empty or blank input is not added	Passed
Project 4	Empty city shows “Please enter a city name!”	Passed
Project 4	Valid city shows temperature, humidity, wind and condition	Passed
Project 4	Unknown city shows “City not found or API error!”	Passed
Project 5	Category buttons and search box filter the product grid	Passed
Project 5	Cart counter shows 3 and total shows ₹85,997 after adding iPhone 16 once and Headphones twice	Passed

4.7 Challenges Faced and Solutions

Table 4.2 – Difficulties and how they were solved

Challenge	Solution
Styles did not load because the link in the HTML pointed to a folder that did not exist (for example css/styles.css)	Checked the relative path in the browser Network panel and kept the folder structure identical to the paths written in the HTML
A navigation link opened a missing page because the file name and the link differed (for example project.html and projects.html)	Used one consistent file name for every page and tested every link after each change
A page looked fine on a laptop but broke on a phone	Added the viewport meta tag and media queries and tested all widths in the Chrome device mode
Tasks disappeared when the To-Do page was refreshed	Saved the tasks array in localStorage after every change and loaded it again at start-up
The weather request failed for a wrongly spelled city and left the page blank	Checked response.ok, threw an Error and showed the message inside a try…catch block
The API key must not be shared in public code	Kept a placeholder in the shared file and used the real key only on the local machine
A missing closing tag disturbed the navigation list of the ShopEase page	Validated the HTML structure with the browser Elements panel and corrected the tag

CHAPTER 5
RESULTS AND DISCUSSION

This chapter presents the outcomes of the internship and discusses what they show.

5.1 Projects Developed

Table 5.1 – Summary of the projects developed

Project	Purpose	Main technologies
1. Accessible Portfolio	Semantic, SEO-friendly and keyboard-accessible personal site	HTML5, ARIA, CSS3
2. Responsive Portfolio	Five-page portfolio that adapts to all screens with a dark mode	CSS Grid, Flexbox, variables, JavaScript
3. To-Do List	Task manager with filtering and saved data	JavaScript, DOM, localStorage
4. Weather Dashboard	Live weather of any city from a REST API	fetch(), async/await, JSON
5. ShopEase	Online shop with products, search, categories and cart	JavaScript modules, CSS, HTML5

All five projects were completed with consistent folder structures and were submitted as separate project folders. Together they form a portfolio that shows the progress from a static page to a data-driven application.

5.2 Features Implemented

- Semantic landmarks, ARIA labels, a skip link and visible keyboard focus.
- SEO meta tags with a description, keywords, author and title for each page.
- Responsive layouts with CSS Grid, Flexbox and media queries.
- A dark and light theme controlled by CSS variables and saved in the browser.
- Create, edit, delete, complete and filter operations for tasks with localStorage persistence.
- Live weather display with input validation and error handling.
- Product grid with search, category filters, a cart sidebar with quantity and total, and a checkout message.

5.3 Skills Acquired

Table 5.2 – Skills gained during the internship

Area	Skills gained
Web structure	Semantic HTML5, forms, landmarks, SEO tags
Accessibility	ARIA attributes, skip links, focus styles, labelled inputs
Styling	CSS3, Flexbox, Grid, variables, media queries, transitions
Programming	JavaScript ES6, arrays and objects, DOM manipulation, event delegation

Area	Skills gained
Data and APIs	localStorage, fetch(), async/await, JSON parsing, error handling
Application design	Modular code, state management, search and filter logic, cart calculation
Professional	Time management, testing, debugging, documentation and submission of assignments

5.4 Comparison of the Five Projects

Table 5.3 – Growth in complexity across the projects

Aspect	Projects 1–2	Projects 3–4	Project 5
Main focus	Structure, accessibility and layout	Logic, storage and API data	Complete application
JavaScript use	None to a small toggle	Core of the project	Several modules
Data source	Written in HTML	Browser storage / REST API	JavaScript array
User input	Contact form	Text field and buttons	Search, filters and cart
Error handling	Browser form validation	Empty input and failed requests	Empty cart message

The comparison shows how each project increased the level of difficulty and reused the earlier skills. The last project needed the layout skills of Project 2, the state handling of Project 3 and the data handling of Project 4 at the same time.

5.5 Internship Experience

The internship at Thiranex was a practical introduction to professional front-end development. Working in the online mode needed self-discipline: the intern had to plan the work of each assignment, find and read documentation, and correct mistakes without a teacher at the side. The projects were evaluated by the output on the screen, so every detail – a wrong path, a missing closing tag or a missing check for empty input – had to be found and corrected before the assignment was complete.

5.6 Discussion

The results show that a student who builds complete small projects learns faster than one who only reads about the technologies. Problems such as broken paths, differences between screen sizes and failed network requests are met only when real pages are built, and solving them gave the intern a lasting understanding of how the browser works.

The projects developed are front-end projects. The data of the To-Do List is stored only in one browser, the weather data depends on a third-party API, and the products of ShopEase are written in a JavaScript file instead of a database. These limits are normal for a first internship and show clearly what must be learned next: a back-end, a database and deployment.

CHAPTER 6
CONCLUSION AND FUTURE WORK

6.1 Conclusion of Individual Projects

- Accessible portfolio: a semantic, SEO-ready and keyboard-friendly website was built using landmarks, ARIA attributes, a skip link and an accessible form.
- Responsive portfolio: a five-page website with CSS Grid, Flexbox, variables, media queries and a dark mode adapts to phone, tablet and desktop screens.
- To-Do List: a complete task manager was created with create, edit, delete, complete and filter functions and with data saved in localStorage.
- Weather Dashboard: asynchronous JavaScript was used to read live data from a REST API, with validation and error messages.
- ShopEase: a modular online-shop interface with search, categories, cart and checkout message was developed.

6.2 Overall Conclusion

The Web Development Internship at Thiranex gave the intern a strong practical base in front-end development. Starting from semantic HTML and finishing with a modular e-commerce catalog, the five projects covered structure, style, behaviour, data and application design. The internship improved not only technical skill but also the habits of testing, debugging and delivering finished work, and it prepared the intern for advanced subjects and for further internships.

6.3 Future Work

- Learn a front-end library such as React and rebuild ShopEase as a component-based application.
- Add a back-end with Node.js and Express and a database such as MongoDB or MySQL so that data is shared between users.
- Add user registration and login with secure authentication.
- Connect a payment gateway to the ShopEase checkout and store the orders.
- Deploy the projects on GitHub Pages, Netlify or a cloud service with a public address.
- Extend the Weather Dashboard with a five-day forecast, location detection and charts.
- Add automated tests and a performance and accessibility audit with Lighthouse.

6.4 Skills Acquired

- Technical: HTML5, CSS3, Flexbox, CSS Grid, JavaScript ES6, DOM, localStorage, fetch() and JSON.
- Design: responsive layout, accessible interfaces, colour themes and consistent styling.
- Professional: planning, self-learning, debugging, testing and meeting submission deadlines.

6.5 Limitations of the Present Work

- All projects are front-end only; there is no server, database or authentication.
- The weather data depends on an external API and an API key, and the products and prices of ShopEase are sample data.
- The internship was short (31 days), so each topic was learned to a fundamental level.
- Testing was manual; automated and cross-browser testing were not part of the internship.

6.6 Benefits of the Internship

For the intern, the internship provided hands-on skill, five finished projects for a portfolio, confidence in debugging and a certificate from an industry organisation. For the institution, it supports the aim of making students industry ready through practical learning, in line with the outcomes of the programme.

REFERENCES

[1] Mozilla Developer Network, “MDN Web Docs: HTML – HyperText Markup Language,” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTML

[2] Mozilla Developer Network, “MDN Web Docs: CSS – Cascading Style Sheets,” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS

[3] Mozilla Developer Network, “MDN Web Docs: JavaScript,” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript

[4] W3C Web Accessibility Initiative, “WCAG 2 Overview,” [Online]. Available: https://www.w3.org/WAI/standards-guidelines/wcag/

[5] W3C Web Accessibility Initiative, “WAI-ARIA Authoring Practices Guide,” [Online]. Available: https://www.w3.org/WAI/ARIA/apg/

[6] Mozilla Developer Network, “CSS grid layout” and “CSS flexible box layout,” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_grid_layout

[7] Mozilla Developer Network, “Web Storage API,” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API

[8] Mozilla Developer Network, “Fetch API,” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

[9] OpenWeatherMap, “Current weather data API,” [Online]. Available: https://openweathermap.org/current

[10] J. Duckett, HTML & CSS: Design and Build Websites. Indianapolis, IN, USA: Wiley, 2011.

[11] M. Haverbeke, Eloquent JavaScript, 4th ed. San Francisco, CA, USA: No Starch Press, 2024. [Online]. Available: https://eloquentjavascript.net/

[12] Google Search Central, “Search Engine Optimization (SEO) Starter Guide,” [Online]. Available: https://developers.google.com/search/docs/fundamentals/seo-starter-guide

[13] Google, “Responsive web design basics,” web.dev, [Online]. Available: https://web.dev/articles/responsive-web-design-basics

APPENDIX
INTERNSHIP COMPLETION CERTIFICATE



Certificate of Achievement issued by Thiranex, Founder & CEO Mr. Hariharan M (Certificate ID: THX-AUG0426-138)